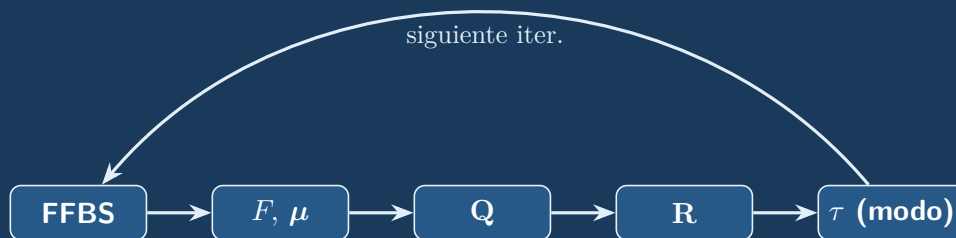


DNS / DNSS Bayesiano con Gibbs Sampler

Version 2 — tau dinamico

Tres modos de modelado de τ - FFBS generalizado
Posteriors conjugadas · MH adaptativo · Diagnosticos MCMC



'estatico'	'ventana'	'ar1'
τ global, MH	τ por ventanas	τ_t AR(1) latente
Mas rapido	Semi-adaptativo	Totalmente dinamico

Contents

1	Introduccion y motivacion	3
1.1	Por que un enfoque bayesiano	3
1.2	Novedad de la version 2: tau dinamico	3
1.3	Estructura del notebook	4
2	Modelo espacio-estado	4
2.1	Formulacion general	4
2.2	Cargas Nelson-Siegel (DNS, $k = 3$)	4
2.3	Cargas Nelson-Siegel-Svensson (DNSS, $k = 4$)	5
2.4	Estructura de $\mathbf{F}$	5
3	Forward Filter Backward Sampler (FFBS)	5
3.1	Por que FFBS y no muestreo univariado	5
3.2	Algoritmo completo	5
3.3	Inicializacion estacionaria	6
3.4	Implementacion	6
4	Distribuciones posteriores conjugadas	6
4.1	Posterior de $\boldsymbol{\mu}$ y f_i (Normal conjugada)	7
4.2	Posterior de q_i (Inverse-Gamma conjugada)	7
4.3	Posterior de r_n (Inverse-Gamma conjugada)	7
4.4	Extensiones para plazos largos	7
4.4.1	Opcion A — $\mathbf{R}$ estructurada por grupos de plazos	7
4.4.2	Opcion C — Pesos en la posterior de $\mathbf{R}$	8
5	Metropolis-Hastings para τ — modo estatico	8
5.1	Log-verosimilitud marginal	8
5.2	Propuesta y ratio de aceptacion	8
5.3	Adaptacion durante el burn-in	9
6	Modos de modelado de τ (novedad v2)	9
6.1	Modo 'ventana' — τ semi-adaptativo	9
6.2	Modo 'ar1' — τ_t latente con dinamica AR(1)	10
6.2.1	Especificacion del proceso	10
6.2.2	Priors sobre los hiperparametros	10
6.2.3	Muestreo de $x_{1:T}$ via MH por periodo	10
6.2.4	Posteriors conjugadas de los hiperparametros	11
6.2.5	Ciclo Gibbs completo con modo AR(1)	11
7	Especificacion de priors	11
8	Diagnosticos de convergencia MCMC	12
8.1	Estadistico $\hat{R}$ de Gelman-Rubin	13
8.2	Effective Sample Size (ESS)	13

8.3	Visualizaciones de diagnostico	13
9	Documentacion de funciones	13
9.1	Cargas	14
9.2	Posteriors conjugadas	14
9.3	Muestreo de τ	14
9.4	Modo ventana	15
9.5	run_chain() v2	16
9.6	Multi-cadena	16
9.7	Diagnosticos	16
9.8	Post-ajuste	17
10	Funcion principal fit_dns_bayes() v2	17
10.1	Firma completa	17
10.2	Descripcion de parametros	18
10.3	Estructura del retorno	19
11	Ejemplos de uso	20
11.1	Ejemplo 1 — DNS basico (tau estatico)	20
11.2	Ejemplo 2 — DNSS con tau AR(1) latente	20
11.3	Ejemplo 3 — DNS con tau por ventanas	21
11.4	Ejemplo 4 — Comparar los tres modos de tau	22
11.5	Ejemplo 5 — DNSS con Opciones A y C + tau ventana	22
11.6	Ejemplo 6 — Comparativa bayesiano vs KF	23
12	Referencia completa de funciones	24
13	Guia de decisiones	25
	Referencias	25

1. Introduccion y motivacion

1.1. Por que un enfoque bayesiano

El estimador de Maxima Verosimilitud (MLE) con Filtro de Kalman produce estimaciones *puntuales* de los factores $\beta_{1:T}$ y los parametros θ . Esto tiene dos consecuencias practicas:

- Los **intervalos de confianza** de los factores son condicionales en $\hat{\theta}$ y subestiman la incertidumbre total.
- El parametro de forma τ se selecciona como un valor fijo (Grid Search), descartando cualquier incertidumbre sobre su valor y su posible variacion temporal.

El enfoque bayesiano resuelve ambos problemas al probar sobre la distribucion posterior conjunta $p(\beta_{1:T}, \theta, \tau | \mathbf{Y})$.

Table 1: Comparacion Kalman Filter + MLE vs Gibbs Sampler bayesiano

Aspecto	Kalman Filter + MLE	Gibbs Sampler (Bayes)
Factores $\beta_{1:T}$	Estimado puntual $E[\beta_t \mathbf{Y}, \hat{\theta}]$	Distribucion completa $p(\beta_{1:T} \mathbf{Y})$
Parametros θ	Optimizacion numerica (L-BFGS-B)	Distribucion posterior $p(\theta \mathbf{Y})$
Incertidumbre	Solo la del estado ($P_{t t}$)	Parametrica + de estado + de τ
Intervalos	Asintóticos, condicionales en $\hat{\theta}$	Intervalos de credibilidad exactos
τ	Fijo via Grid Search (un valor por periodo)	Posterior completa; opcionalmente dinamico
Costo computacional	$O(T \cdot N^2 \cdot k^2)$	$O(n_{\text{iter}} \cdot T \cdot N^2 \cdot k^2)$

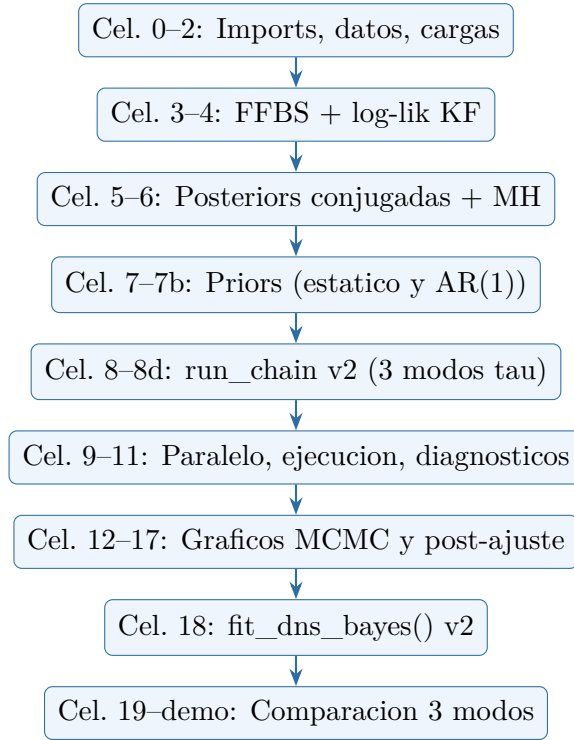
1.2. Novedad de la version 2: tau dinamico

La version 1 del notebook modelaba τ como un escalar global constante a lo largo de toda la muestra. Esto es correcto en muestras cortas o cuando la forma de la curva es estacionaria, pero es restrictivo cuando:

- La politica monetaria cambia de regimen (e.g., TES Colombia 2019–2026).
- El hump de la curvatura se desplaza sistematicamente en el tiempo.
- El analisis cubre periodos con distintas condiciones de liquidez.

La version 2 implementa tres modos de modelado de τ seleccionables mediante el parametro `tau_mode` de `fit_dns_bayes()`.

1.3. Estructura del notebook



2. Modelo espacio-estado

2.1. Formulacion general

Modelo espacio-estado DNS/DNSS

Ecuacion de observacion:

$$\mathbf{y}_t = \mathbf{H}(\boldsymbol{\tau}) \boldsymbol{\beta}_t + \boldsymbol{\varepsilon}_t, \quad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R}) \quad (1)$$

Ecuacion de transicion:

$$\boldsymbol{\beta}_t = \boldsymbol{\mu} + \mathbf{F}(\boldsymbol{\beta}_{t-1} - \boldsymbol{\mu}) + \boldsymbol{\eta}_t, \quad \boldsymbol{\eta}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \quad (2)$$

donde $\mathbf{y}_t \in \mathbb{R}^N$, $\boldsymbol{\beta}_t \in \mathbb{R}^k$ ($k = 3$ DNS, $k = 4$ DNSS), $\mathbf{H}(\boldsymbol{\tau}) \in \mathbb{R}^{N \times k}$, $\mathbf{R} = \text{diag}(r_1, \dots, r_N)$, $\mathbf{Q} = \text{diag}(q_1, \dots, q_k)$.

2.2. Cargas Nelson-Siegel (DNS, $k = 3$)

$$\mathbf{H}_n(\tau_1) = \left[1, \frac{1 - e^{-m_n/\tau_1}}{m_n/\tau_1}, \frac{1 - e^{-m_n/\tau_1}}{m_n/\tau_1} - e^{-m_n/\tau_1} \right] \quad (3)$$

El hump del factor de curvatura $\beta_{2,t}$ ocurre en $m^* \approx 1.793 \tau_1$ anos.

2.3. Cargas Nelson-Siegel-Svensson (DNSS, $k = 4$)

$$\mathbf{H}_n(\tau_1, \tau_2) = \left[\mathbf{H}_n(\tau_1), \frac{1 - e^{-m_n/\tau_2}}{m_n/\tau_2} - e^{-m_n/\tau_2} \right] \quad (4)$$

Restriccion de separacion: $|\tau_1 - \tau_2| \geq 0.15$ para evitar colinealidad.

2.4. Estructura de $\mathbf{F}$

Table 2: Opciones de estructura de $\mathbf{F}$

$\mathbf{F_type}$	Parametros en $\mathbf{F}$	Muestreo	Referencia
'diagonal'	k	Normal conjugada truncada	Caldeira et al. (2010) P7
'triangular'	$k(k+1)/2$	Diagonal conj. + MH off-diag	Ext. propia
'completa'	k^2	MH matricial	VAR(1) irrestricto

3. Forward Filter Backward Sampler (FFBS)

3.1. Por que FFBS y no muestreo univariado

Muestrear cada β_t condicionado en sus vecinos produce cadenas con autocorrelacion extremadamente alta. El FFBS de Carter & Kohn (1994) muestrea la trayectoria *completa* $\beta_{1:T}$ en una sola pasada, eliminando esta autocorrelacion interna.

3.2. Algoritmo completo

Algoritmo 3.1 (FFBS — Carter & Kohn (1994)). Dado $\theta = (\mu, \mathbf{F}, \mathbf{Q}, \mathbf{R}, \tau)$, muestrear $\beta_{1:T} \sim p(\beta_{1:T} \mid \mathbf{Y}, \theta)$.

Fase forward (Kalman Filter): para $t = 1, \dots, T$:

$$\hat{\beta}_{t|t-1} = \mu + \mathbf{F}(\hat{\beta}_{t-1|t-1} - \mu) \quad (5)$$

$$\mathbf{P}_{t|t-1} = \mathbf{F} \mathbf{P}_{t-1|t-1} \mathbf{F}^\top + \mathbf{Q} \quad (6)$$

$$\mathbf{v}_t = \mathbf{y}_t - \mathbf{H} \hat{\beta}_{t|t-1} \quad (7)$$

$$\mathbf{S}_t = \mathbf{H} \mathbf{P}_{t|t-1} \mathbf{H}^\top + \mathbf{R} \quad (8)$$

$$\mathbf{K}_t = \mathbf{P}_{t|t-1} \mathbf{H}^\top \mathbf{S}_t^{-1} \quad (9)$$

$$\hat{\beta}_{t|t} = \hat{\beta}_{t|t-1} + \mathbf{K}_t \mathbf{v}_t \quad (10)$$

$$\mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t \mathbf{H}) \mathbf{P}_{t|t-1} (\mathbf{I} - \mathbf{K}_t \mathbf{H})^\top + \mathbf{K}_t \mathbf{R} \mathbf{K}_t^\top \quad (\text{forma Joseph}) \quad (11)$$

Fase backward: muestrear $\beta_T \sim \mathcal{N}(\hat{\beta}_{T|T}, \mathbf{P}_{T|T})$. Para $t = T - 1, \dots, 1$:

$$\mathbf{P}_{t+1|t} = \mathbf{F} \mathbf{P}_{t|t} \mathbf{F}^\top + \mathbf{Q} \quad (12)$$

$$\mathbf{G}_t = \mathbf{P}_{t|t} \mathbf{F}^\top \mathbf{P}_{t+1|t}^{-1} \quad (\text{ganancia suavizadora}) \quad (13)$$

$$\boldsymbol{\mu}_{t|T} = \hat{\beta}_{t|t} + \mathbf{G}_t (\beta_{t+1} - \hat{\beta}_{t+1|t}) \quad (14)$$

$$\boldsymbol{\Sigma}_{t|T} = \mathbf{P}_{t|t} - \mathbf{G}_t \mathbf{P}_{t+1|t} \mathbf{G}_t^\top \quad (15)$$

$$\beta_t \sim \mathcal{N}(\boldsymbol{\mu}_{t|T}, \boldsymbol{\Sigma}_{t|T}) \quad (16)$$

Forma Joseph (ecuacion 11)

La actualizacion de $\mathbf{P}_{t|t}$ usa la forma Joseph $(I - KH)P(I - KH)^\top + KRK^\top$ en lugar de la forma estandar $(I - KH)P$. Garantiza simetria y positividad definida ante errores de redondeo acumulados.

3.3. Inicializacion estacionaria

Para $\mathbf{F}$ diagonal, la condicion inicial $\mathbf{P}_{0|0}$ se obtiene directamente:

$$P_{0,ii} = \frac{q_i}{1 - f_i^2}, \quad i = 1, \dots, k \quad (17)$$

3.4. Implementacion

La funcion `_ffbs_diag()` esta compilada con Numba JIT para $\mathbf{F}$ diagonal y es valida para $k = 3$ (DNS) y $k = 4$ (DNSS) sin modificar el codigo. El wrapper publico `ffbs()` garantiza los tipos correctos:

Celda 3 --- ffbs() (wrapper publico)

```
1 def ffbs(mu, f_vec, q_vec, r_vec, H, yields):
2     '''Wrapper FFBS para F diagonal. Valido k=3 (DNS) y k=4 (DNSS).'''
3     return _ffbs_diag(
4         np.asarray(mu, dtype=np.float64),
5         np.asarray(f_vec, dtype=np.float64), # diag(F): (k,)
6         np.asarray(q_vec, dtype=np.float64), # diag(Q): (k,)
7         np.asarray(r_vec, dtype=np.float64), # diag(R): (N,)
8         np.asarray(H, dtype=np.float64), # cargas: (N, k)
9         np.asarray(yields, dtype=np.float64), # datos: (T, N)
10    )
11    # Retorna: betas (T, k) --- una trayectoria de la posterior
```

4. Distribuciones posteriores conjugadas

Condicionado en $\beta_{1:T}$ (muestreado por FFBS), todos los demas parametros tienen posteriors analiticas exactas.

4.1. Posterior de μ y f_i (Normal conjugada)

Posterior de μ_i dado f_i

Con prior $\mu_i \sim \mathcal{N}(m_{0i}, V_{0i})$, definiendo $\ell_{i,t} = \beta_{i,t} - f_i \beta_{i,t-1}$:

$$\mu_i \mid f_i, \boldsymbol{\beta}, q_i \sim \mathcal{N}(\bar{m}_i, \bar{V}_i) \quad (18)$$

$$\bar{V}_i^{-1} = \frac{1}{V_{0i}} + \frac{(1 - f_i)^2 T_{\text{eff}}}{q_i}, \quad \bar{m}_i = \bar{V}_i \left(\frac{m_{0i}}{V_{0i}} + \frac{1 - f_i}{q_i} \sum_t \ell_{i,t} \right) \quad (19)$$

donde $T_{\text{eff}} = T - 1$.

Posterior de f_i dado μ_i (Normal truncada)

Definiendo $a_{i,t} = \beta_{i,t} - \mu_i$:

$$f_i \mid \mu_i, \boldsymbol{\beta}, q_i \sim \mathcal{TN}_{(-1,1)}(\bar{f}_i, \bar{V}_{f_i}) \quad (20)$$

$$\bar{V}_{f_i}^{-1} = \frac{1}{V_{f_{0i}}} + \frac{\sum_t a_{i,t}^2}{q_i}, \quad \bar{f}_i = \bar{V}_{f_i} \left(\frac{f_{0i}}{V_{f_{0i}}} + \frac{\sum_t a_{i,t} a_{i,t-1}}{q_i} \right) \quad (21)$$

La truncacion en $(-1, 1)$ garantiza estacionariedad del AR(1) de cada factor.

Observacion 4.1. El muestreo es secuencial: $\mu_i \rightarrow f_i$ por cada factor $i = 1, \dots, k$. Esto es Gibbs dentro de Gibbs y garantiza consistencia interna en cada iteracion.

4.2. Posterior de q_i (Inverse-Gamma conjugada)

Con residuos de estado $\eta_{i,t} = \beta_{i,t} - \mu_i - f_i(\beta_{i,t-1} - \mu_i)$:

$$q_i \mid \boldsymbol{\beta}, \mu_i, f_i \sim \mathcal{IG} \left(a_0 + \frac{T_{\text{eff}}}{2}, b_0 + \frac{\sum_t \eta_{i,t}^2}{2} \right) \quad (22)$$

4.3. Posterior de r_n (Inverse-Gamma conjugada)

Con residuos de medicion $\varepsilon_{n,t} = y_{t,n} - \mathbf{H}_n \boldsymbol{\beta}_t$:

$$r_n \mid \boldsymbol{\beta}, \mathbf{Y} \sim \mathcal{IG} \left(a_0 + \frac{T}{2}, b_0 + \frac{\sum_t \varepsilon_{n,t}^2}{2} \right) \quad (23)$$

4.4. Extensiones para plazos largos

4.4.1. Opcion A — R estructurada por grupos de plazos

En lugar de estimar N varianzas independientes, se reducen a 3 parametros por grupos de plazos (corto $\leq \text{th1}$, medio $\leq \text{th2}$, largo $> \text{th2}$). La posterior por grupo $\mathcal{G}_g$ recoge todos sus residuos:

$$r_g \mid \beta, \mathbf{Y} \sim \mathcal{IG}\left(a_0 + \frac{T|\mathcal{G}_g|}{2}, b_0 + \frac{\sum_{n \in \mathcal{G}_g} \sum_t \varepsilon_{n,t}^2}{2}\right) \quad (24)$$

4.4.2. Opcion C — Pesos en la posterior de R

Asignar peso $w_n > 1$ a un plazo n reduce el b_0 efectivo, concentrando la posterior en valores menores de r_n (mejor ajuste):

$$r_n^{(w)} \mid \beta, \mathbf{Y} \sim \mathcal{IG}\left(a_0 + \frac{T}{2}, \frac{b_0}{w_n} + \frac{\sum_t \varepsilon_{n,t}^2}{2}\right) \quad (25)$$

5. Metropolis-Hastings para τ — modo estatico

5.1. Log-verosimilitud marginal

τ entra de forma no lineal en $\mathbf{H}(\tau)$, por lo que no existe familia conjugada. Se usa MH con la log-verosimilitud *marginal* $p(\mathbf{Y} \mid \tau, \boldsymbol{\theta})$ donde $\beta_{1:T}$ esta integrado via KF:

$$\log p(\mathbf{Y} \mid \tau, \boldsymbol{\theta}) = -\frac{TN}{2} \log(2\pi) - \frac{1}{2} \sum_{t=1}^T [\log |\mathbf{S}_t| + \mathbf{v}_t^\top \mathbf{S}_t^{-1} \mathbf{v}_t] \quad (26)$$

El $\log |\mathbf{S}_t|$ se calcula via Cholesky para estabilidad numerica.

Por que log-verosimilitud marginal y no condicional en β

Si se evaluara la log-lik con la trayectoria $\beta_{1:T}$ del paso anterior (que fue muestreada con el τ antiguo), la comparacion seria inconsistente: β y τ^* usarian parametros de forma distintos. La log-lik marginal promedia correctamente sobre todas las trayectorias posibles, garantizando que el ratio de aceptacion sea valido.

5.2. Propuesta y ratio de aceptacion

MH para τ — propuesta random walk en log-escala

Para garantizar positividad sin truncar el espacio:

$$\text{DNS: } \log \tau_1^* = \log \tau_1 + \sigma_1 z, \quad z \sim \mathcal{N}(0, 1) \quad (27)$$

$$\text{DNSS: } \begin{pmatrix} \log \tau_1^* \\ \log \tau_2^* \end{pmatrix} = \begin{pmatrix} \log \tau_1 \\ \log \tau_2 \end{pmatrix} + \begin{pmatrix} \sigma_1 z_1 \\ \sigma_2 z_2 \end{pmatrix}, \quad z_1, z_2 \stackrel{iid}{\sim} \mathcal{N}(0, 1) \quad (28)$$

Ratio de aceptacion ($\alpha = \min(1, e^{\log \alpha})$):

$$\log \alpha = \underbrace{\log p(\mathbf{Y} \mid \tau^*, \boldsymbol{\theta}) - \log p(\mathbf{Y} \mid \tau, \boldsymbol{\theta})}_{\Delta \log\text{-lik}} + \underbrace{\log p(\tau^*) - \log p(\tau)}_{\Delta \log\text{-prior}} \quad (29)$$

El Jacobiano del cambio de variables $\log \tau \rightarrow \tau$ cancela porque la propuesta es simetrica en log-escala.

5.3. Adaptacion durante el burn-in

La std de la propuesta σ se ajusta cada $n_{\text{burnin}}/5$ iteraciones:

$$\sigma \leftarrow \begin{cases} 0.70 \sigma & \text{si } \hat{r} < r_{\text{target}} - 0.12 \\ 1.35 \sigma & \text{si } \hat{r} > r_{\text{target}} + 0.15 \\ \sigma & \text{en otro caso} \end{cases} \quad (30)$$

con $r_{\text{target}} = 0.40$ (DNS, 1D) y $r_{\text{target}} = 0.28$ (DNSS, 2D), y σ acotado en $[0.005, 0.60]$.

6. Modos de modelado de τ (novedad v2)

La version 2 permite elegir entre tres modos de modelado de τ mediante el parametro `tau_mode`. El modo 'estatico' (Seccion 5) es el comportamiento clasico de la v1. Los modos 'ventana' y 'ar1' permiten capturar variacion temporal de la forma de la curva.

Table 3: Comparativa de los tres modos de tau

Modo	Descripcion	Ventajas	Limitaciones
'estatico'	τ global unico para toda la muestra. Posterior via MH estandar.	Mas rapido; IC de τ bien definido; minimo de parametros.	Asume forma de curva constante en t .
'ventana'	τ cambia entre ventanas rodantes de tamaño W . Mini-sampler por ventana.	Semi-adaptativo sin cambio de modelo; facil de interpretar.	IC de τ mas amplios por menor informacion en cada ventana.
'ar1'	τ_t sigue AR(1) latente en log-escala con hiperparametros ϕ_x, σ_x .	Totalmente dinamico; captura cambios de regimen de forma continua.	El mas lento; requiere mas iteraciones para convergencia.

6.1. Modo 'ventana' — τ semi-adaptativo

La muestra se divide en ventanas solapadas de tamaño W con desplazamiento s :

$$\mathcal{W}_j = [j \cdot s, j \cdot s + W - 1], \quad j = 0, 1, \dots, \left\lfloor \frac{T - W}{s} \right\rfloor \quad (31)$$

En cada ventana se corre un mini-sampler de 5 pasos de MH con la log-lik local $p(\mathbf{Y}_{\mathcal{W}_j} | \tau, \boldsymbol{\theta})$. La secuencia temporal τ_t se ensambla promediando las estimaciones de todas las ventanas que contienen el periodo t .

Eleccion de W

Para datos mensuales de TES Colombia, se recomienda $W \approx 36\text{--}60$ meses (3–5 años). Ventanas mas grandes producen τ_t mas suaves; ventanas menores capturan cambios mas rapidos pero con mayor varianza.

6.2. Modo 'ar1' — τ_t latente con dinamica AR(1)**6.2.1. Especificacion del proceso**

Se define $x_t = \log \tau_{1,t}$ (y $z_t = \log \tau_{2,t}$ para DNSS) como proceso AR(1) latente en log-escala:

$$x_t = \mu_x + \phi_x(x_{t-1} - \mu_x) + \sigma_x \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1) \quad (32)$$

La log-escala garantiza $\tau_t = e^{x_t} > 0$ en todo momento. La media $\mu_x = \log(\tau_{\text{init}})$ se fija en la inicializacion OLS.

6.2.2. Priors sobre los hiperparametros

$$\phi_x \sim \mathcal{TN}_{(0,1)}(0.95, 0.05^2) \quad (33)$$

$$\sigma_x^2 \sim \mathcal{IG}(2, 0.01) \quad (34)$$

El prior de ϕ_x centrado en 0.95 refleja que τ_t cambia lentamente (alta persistencia). El prior de σ_x^2 muy pequeno evita cambios abruptos.

6.2.3. Muestreo de $x_{1:T}$ via MH por periodo

Para cada $t = 1, \dots, T$, se propone $x_t^* = x_t + \sigma_{\text{prop}} z$ y se acepta con ratio:

$$\log \alpha_t = \underbrace{\log p(\mathbf{y}_t | x_t^*, \boldsymbol{\beta}_t, \mathbf{R}) - \log p(\mathbf{y}_t | x_t, \boldsymbol{\beta}_t, \mathbf{R})}_{\Delta \log\text{-lik local}} + \underbrace{\log p(x_t^* | x_{t-1}, x_{t+1}) - \log p(x_t | x_{t-1}, x_{t+1})}_{\Delta \log\text{-prior condicional}} \quad (35)$$

La log-lik local para el periodo t es:

$$\log p(\mathbf{y}_t | x_t, \boldsymbol{\beta}_t, \mathbf{R}) = -\frac{1}{2} \sum_{n=1}^N \left[\frac{(y_{t,n} - \mathbf{H}_n(e^{x_t}) \boldsymbol{\beta}_t)^2}{r_n} + \log(2\pi r_n) \right] \quad (36)$$

La prior condicional de x_t dado sus vecinos es:

$$\log p(x_t | x_{t-1}, x_{t+1}) \propto -\frac{1}{2\sigma_x^2} \left[(x_t - \mu_x - \phi_x(x_{t-1} - \mu_x))^2 + (x_{t+1} - \mu_x - \phi_x(x_t - \mu_x))^2 \right] \quad (37)$$

6.2.4. Posteriors conjugadas de los hiperparametros

Posterior de ϕ_x dado $x_{1:T}$

Con $a_t = x_t - \mu_x$ y prior $\phi_x \sim \mathcal{TN}_{(0,1)}(\bar{\phi}, V_\phi)$:

$$\phi_x \mid x_{1:T}, \sigma_x \sim \mathcal{TN}_{(0,1)}\left(\frac{\bar{\phi}/V_\phi + \sum_t a_{t-1}a_t/\sigma_x^2}{1/V_\phi + \sum_t a_{t-1}^2/\sigma_x^2}, \left[\frac{1}{V_\phi} + \frac{\sum_t a_{t-1}^2}{\sigma_x^2}\right]^{-1}\right) \quad (38)$$

Posterior de σ_x^2 dado $x_{1:T}$

Con residuos $\eta_t = x_t - \mu_x - \phi_x(x_{t-1} - \mu_x)$:

$$\sigma_x^2 \mid x_{1:T}, \phi_x \sim \mathcal{IG}\left(a_0 + \frac{T-1}{2}, b_0 + \frac{\sum_{t=2}^T \eta_t^2}{2}\right) \quad (39)$$

6.2.5. Ciclo Gibbs completo con modo AR(1)

Algoritmo 6.1 (Gibbs con τ_t AR(1)). Por cada iteracion s :

1. $\beta_{1:T} \mid \bar{\tau}, \theta \rightarrow \mathbf{FFBS}$ con $\mathbf{H}(\bar{\tau})$, donde $\bar{\tau} = \text{mediana}(e^{x_{1:T}})$
2. $\mathbf{F}, \mu \mid \beta, \mathbf{Q} \rightarrow$ Normal conjugada truncada (diagonal)
3. $\mathbf{Q} \mid \beta, \mathbf{F}, \mu \rightarrow$ Inverse-Gamma conjugada
4. $\mathbf{R} \mid \beta, \mathbf{Y} \rightarrow$ IG conjugada (libre, grupos, o ponderada)
5. $x_{1:T} \mid \beta, \theta, \phi_x, \sigma_x \rightarrow$ MH por periodo (ec. 35)
6. $\phi_x \mid x_{1:T}, \sigma_x \rightarrow$ Normal truncada conjugada (ec. 38)
7. $\sigma_x^2 \mid x_{1:T}, \phi_x \rightarrow$ Inverse-Gamma conjugada (ec. 39)

Calibracion de tau_ar1_prop_std

La std de propuesta por periodo `tau_ar1_prop_std` controla la tasa de aceptacion del MH local. Se recomienda comenzar con 0.04–0.06 y ajustar para obtener una tasa media entre 30% y 55%. Una tasa muy alta indica que τ_t varia poco y el modo 'estatico' puede ser suficiente.

7. Especificacion de priors

Los priors siguen la filosofia debilmente informativos de Caldeira et al. (2010) y Cakmakli (2013).

Table 4: Especificacion de priors — funcion `make_priors()`

Parametro	Prior	Justificacion
f_i	$\mathcal{TN}_{(-1,1)}(0.90, 0.10^2)$	Alta persistencia tipica en DNS; truncacion garantiza estacionariedad
μ_i	$\mathcal{N}(m_{0i}, 25)$	Difuso; centrado en valores historicos plausibles
q_i, r_n	$\mathcal{IG}(0.01, 0.01)$	Casi no-informativo (Jeffreys-like)
τ (uniforme)	$\mathcal{U}(0.15, 5.0)$	Bounds economicos; hump en $[0.27, 9.0]$ anos
τ_1 (normal)	$\mathcal{N}(1.78, 0.40^2)$	Prior informativo; hump esperado en ≈ 3.2 anos
τ_2 (normal)	$\mathcal{N}(0.60, 0.08^2)$	Fuertemente informativo; evita colinealidad con τ_1
ϕ_x (AR(1) de τ)	$\mathcal{TN}_{(0,1)}(0.95, 0.05^2)$	Alta persistencia de τ_t ; solo para <code>tau_mode='ar1'</code>
σ_x^2 (AR(1) de τ)	$\mathcal{IG}(2, 0.01)$	Varianza pequena; τ_t cambia lentamente

Celda 7 / 7b --- `make_priors()` y `make_priors_ar1()`

```

1 # Prior uniforme (difuso, modo estatico o ventana)
2 priors = make_priors('DNSS', prior_tau_type='uniforme',
3                     tau_lo=0.15, tau_hi=5.0)
4
5 # Prior normal informativo (cuando se conoce el regimen)
6 priors = make_priors('DNSS', prior_tau_type='normal',
7                     tau1_mu=1.78, tau1_sd=0.40,
8                     tau2_mu=0.60, tau2_sd=0.08)
9
10 # Priors extendidos para modo AR(1)
11 priors = make_priors_ar1('DNS', tau1_init=1.78,
12                         phi_x_mean=0.95, phi_x_sd=0.05,
13                         sigma_x_a=2.0, sigma_x_b=0.01)

```

8. Diagnosticos de convergencia MCMC

8.1. Estadístico $\hat{R}$ de Gelman-Rubin

$\hat{R}$ de Gelman-Rubin (1992)

Con m cadenas de longitud n para un parametro escalar ψ :

$$B = \frac{n}{m-1} \sum_{j=1}^m (\bar{\psi}_j - \bar{\psi})^2 \quad (40)$$

$$W = \frac{1}{m} \sum_{j=1}^m s_j^2 \quad (41)$$

$$\hat{R} = \sqrt{\frac{(n-1)W/n + B/n}{W}} \quad (42)$$

Criterios: $\hat{R} < 1.05$ (excelente), $\hat{R} < 1.10$ (aceptable), $\hat{R} \geq 1.10$ (no convergio — extender iteraciones).

8.2. Effective Sample Size (ESS)

$$\text{ESS} = \frac{n}{1 + 2 \sum_{k=1}^{\infty} \hat{\rho}_k} \quad (43)$$

La suma se trunca cuando $|\hat{\rho}_k| < 0.05$. Se considera aceptable $\text{ESS} > 400$ y pobre $\text{ESS} < 100$.

8.3. Visualizaciones de diagnostico

El notebook genera tres bloques de diagnostico MCMC:

1. **Traceplots** (`plot_traceplots`): evolucion de los parametros por iteracion. Una cadena bien mezclada explora el espacio, se solapa entre cadenas y no tiene tendencia sistematica.
2. **ACF de la cadena** (`plot_acf_mcmc`): funcion de autocorrelacion de las muestras con ESS por parametro.
3. **Distribuciones posteriores** (`plot_posteriors`): histogramas con KDE, media posterior e IC 95%.

9. Documentacion de funciones

9.1. Cargas

Celda 2 --- cargas NS y NSS

```

1 def ns_loadings(maturities, tau1):
2     '''Nelson-Siegel. Shape (N, 3): [nivel, pendiente, curvatura].'''
3     ...
4
5 def nss_loadings(maturities, tau1, tau2):
6     '''Nelson-Siegel-Svensson. Shape (N, 4).
7     Condicion: |tau1 - tau2| >= 0.15 para evitar colinealidad.'''
8     ...
9
10 def get_H(maturities, tau1, tau2, model):
11     '''Wrapper: model='DNS' -> ns_loadings | 'DNSS' -> nss_loadings.'''
12     ...

```

9.2. Posteriors conjugadas

Celda 5 --- muestreo de F, mu, Q, R

```

1 # F diagonal y mu --- Normal conjugada
2 f_new, mu_new = sample_F_mu_diagonal(
3     betas,                                # (T, k) trayectoria FFBS actual
4     f_vec, q_vec,                        # parametros actuales
5     prior_f_mean, prior_f_var,          # prior de f_i
6     prior_mu_mean, prior_mu_var)        # prior de mu_i
7
8 # Q --- Inverse-Gamma por factor
9 q_new = sample_Q(betas, mu, f_vec, prior_Q_a, prior_Q_b)
10
11 # R --- libre: N varianzas independientes
12 r_new = sample_R(yields, betas, H, prior_R_a, prior_R_b,
13                 obs_weights=None)      # Opcion C: pesos por plazo
14
15 # R --- grupos: 3 varianzas por tramo (Opcion A)
16 r_new = sample_R_groups(yields, betas, H, prior_R_a, prior_R_b,
17                         maturities, group_thresholds=(3.0, 10.0))

```

9.3. Muestreo de τ

Celda 6 --- MH para tau (modo estatico)

```

1 # DNS: MH 1D para tau1
2 tau1, loglik_new, accepted = sample_tau_MH_dns(
3     tau1, yields, mu, f_vec, q_vec, r_vec,
4     maturities,
5     prop_std = 0.08,

```

```

6     tau_bounds= (0.15, 5.0),
7     prior_tau = ('uniforme', 0.15, 5.0))
8
9 # DNSS: MH 2D para (tau1, tau2)
10 tau1, tau2, loglik_new, accepted = sample_tau_MH_dnss(
11     tau1, tau2, yields, mu, f_vec, q_vec, r_vec,
12     maturities,
13     prop_std = np.array([0.08, 0.06]),
14     tau_bounds = (0.15, 5.0),
15     prior_tau = ('normal', 1.78, 0.40, 0.60, 0.08),
16     sep_min = 0.15)

```

Celda 8b --- muestreo de tau AR(1)

```

1 # Muestrea  $x_{1:T} = \log(\tau_t)$  via MH por periodo
2 x_seq, acc_rate = sample_tau_ar1_MH(
3     x_seq,          # (T, n_tau) secuencia actual de  $\log(\tau_t)$ 
4     betas,          # (T, k) trayectoria FFBS actual
5     yields,         # (T, N)
6     r_vec,          # (N,) varianzas de medicion actuales
7     model, maturities,
8     phi_x,          # float o (n_tau,) persistencia AR(1)
9     mu_x,           # float o (n_tau,) media de largo plazo
10    sigma_x,         # float o (n_tau,) std de innovaciones
11    prop_std_t=0.05)
12 # Retorna: x_seq actualizado (T, n_tau), acc_rate float
13
14 # Muestrea hiperparametros del proceso AR(1)
15 phi_x = sample_phi_x(x_seq, mu_x, sigma_x,
16                      phi_x_mean=0.95, phi_x_var=0.05**2)
17 sigma_x = sample_sigma_x(x_seq, mu_x, phi_x,
18                          sigma_x_a=2.0, sigma_x_b=0.01)

```

9.4. Modo ventana

Celda 8c --- run_rolling_bayes() y assemble_rolling_tau()

```

1 # Corre el Gibbs en ventanas rodantes
2 rolling_res = run_rolling_bayes(
3     yields, maturities, dates, priors,
4     n_iter=2000, n_burnin=600,
5     window_size=52,    # tamaño de ventana en periodos
6     step=None,         # desplazamiento (None -> window_size//2)
7     n_chains=2)
8
9 # Ensambla secuencia tau_t de longitud T
10 tau1_t, tau1_lo, tau1_hi, \
11 tau2_t, tau2_lo, tau2_hi = assemble_rolling_tau(rolling_res, T)

```


9.5. run_chain() v2

Celda 8d --- run_chain() con tau_mode

```

1 store = run_chain(
2     yields, maturities, priors, n_iter, n_burnin, seed,
3     tau_mode      = 'ar1',      # 'estatico' / 'ventana' / 'ar1'
4     tau_window_size = None,      # solo para 'ventana'
5     tau_ar1_prop_std = 0.05,     # solo para 'ar1'
6     obs_weights     = None,      # Opcion C
7     r_mode          = 'libre',   # 'libre' / 'grupos' (Opcion A)
8     group_thresholds = (3.0, 10.0),
9 )
10
11 # Claves adicionales en store para tau_mode='ar1':
12 store['tau1_seq']    # (n_store, T) trayectorias tau1_t
13 store['phi_x1']      # (n_store,) muestras de phi_x
14 store['sigma_x1']    # (n_store,) muestras de sigma_x
15 # Para DNSS tambien: tau2_seq, phi_x2, sigma_x2

```

9.6. Multi-cadena

Celda 9 --- run_chains_parallel() v2

```

1 chains = run_chains_parallel(
2     yields, maturities, priors, n_iter, n_burnin,
3     n_chains      = 4,
4     n_jobs        = -1,          # todos los nucleos disponibles
5     tau_mode      = 'ar1',
6     tau_window_size = None,
7     tau_ar1_prop_std = 0.05,
8     obs_weights     = None,
9     r_mode          = 'libre',
10    group_thresholds = (3.0, 10.0),
11 )

```

9.7. Diagnosticos

Celda 9 --- diagnosticos de convergencia

```

1 # Tabla R-hat y ESS para todos los parametros
2 df_diag = compute_diagnostics(chains)
3 print_diagnostics(chains)
4
5 # R-hat para un parametro especifico
6 rhat_tau1 = gelman_rubin(chains, 'tau1')
7 rhat_f0    = gelman_rubin(chains, 'f_vec', idx=0)
8 # Para AR(1): R-hat de hiperparametros

```

```

9 rhat_phi = gelman_rubin(chains, 'phi_x1')
10 rhat_sig = gelman_rubin(chains, 'sigma_x1')
11
12 # ESS de una cadena
13 ess = effective_sample_size(chains[0]['tau1'])

```

9.8. Post-ajuste

Celdas 15-17 --- factores y residuos

```

1 # Estadísticos de la distribución posterior de beta_t
2 beta_mean, beta_q025, beta_q975, beta_std = \
3     extract_posterior_betas(chains)
4 # beta_mean : (T, k) media posterior E[beta_t | Y]
5 # beta_q025 : (T, k) percentil 2.5%
6 # beta_q975 : (T, k) percentil 97.5%
7
8 # Graficos de factores con bandas de credibilidad
9 plot_factores_bayes(chains, dates)
10
11 # Ajuste in-sample con media posterior
12 fit = compute_posterior_fit(chains, yields, maturities)
13 print(f"RMSE = {fit['rmse_total']*100:.3f} pb")
14
15 # Analisis de residuos
16 analisis_residuos_b(fit)
17 plot_heatmap_residuos_b(fit)
18
19 # Visualizar evolucion temporal de tau (todos los modos)
20 plot_tau_temporal({'Mi modelo': res}, dates, maturities)

```

10. Funcion principal fit_dns_bayes() v2

10.1. Firma completa

Celda 18 --- firma de fit_dns_bayes() v2

```

1 def fit_dns_bayes(
2     dates,                # DatetimeIndex o array de fechas
3     yields,               # (T, N) DataFrame o ndarray
4     maturities,           # (N,) plazos en anos
5     # Modelo
6     model                  = 'DNS',        # 'DNS' (k=3) o 'DNSS' (k=4)
7     # MCMC
8     n_iter                  = 3000,

```

```

9     n_burnin          = 1000,
10    n_chains          = 4,
11    # Modo de tau (NUEVO v2)
12    tau_mode           = 'estatico', # 'estatico' / 'ventana' / 'ar1'
13    tau_window_size    = None,      # solo para tau_mode='ventana'
14    tau_ar1_prop_std   = 0.05,      # solo para tau_mode='ar1'
15    # Priors de tau
16    prior_tau_type     = 'uniforme', # 'uniforme' o 'normal'
17    tau1_mu=1.78, tau1_sd=0.40,
18    tau2_mu=0.60, tau2_sd=0.08,
19    tau_lo=0.15, tau_hi=5.0,
20    # MH
21    prop_std_init      = None,
22    # Mejoras R
23    obs_weights        = None,      # (N,) Opcion C
24    r_mode              = 'libre',   # 'libre' / 'grupos' (Opcion A)
25    group_thresholds   = (3.0, 10.0),
26    # Control
27    plot               = True,
28    verbose             = True,
29    n_jobs              = -1,
30 )

```

10.2. Descripcion de parametros

Parametro	Default	Descripcion
model	'DNS'	'DNS' para Nelson-Siegel ($k = 3$) o 'DNSS' para Svensson ($k = 4$).
n_iter	3000	Total de iteraciones MCMC (incluyendo burn-in). Para produccion: 5,000–10,000.
n_burnin	1000	Iteraciones descartadas. Tipicamente $\sim 30\%$ del total.
n_chains	4	Cadenas paralelas. Minimo 2 para calcular $\hat{R}$.
tau_mode	'estatico'	NUEVO v2. 'estatico': un τ global para toda la muestra (v1). 'ventana': τ semi-adaptativo por ventanas. 'ar1': τ_t AR(1) latente en log-escala. Ver Seccion 6.
tau_window_size	None	Tamano de ventana para tau_mode='ventana'. None $\rightarrow \max(30, T//4)$.
tau_ar1_prop_std	0.05	Std de la propuesta MH por periodo para $\log(\tau_t)$ en modo AR(1). Ajustar para lograr tasa de aceptacion 30–55%.

Parametro	Default	Descripcion
prior_tau_type	'uniforme'	'uniforme': prior plano en (τ_{lo} , τ_{hi}). 'normal': prior normal informativo sobre τ .
obs_weights	None	Opcion C. Array (N ,) de pesos por plazo para la posterior de R . None = todos iguales. Ejemplo: <code>np.where(maturities>=10, 2.0, 1.0)</code> .
r_mode	'libre'	Opcion A. 'libre': N varianzas independientes. 'grupos': 3 varianzas por grupos de plazos.
group_thresholds	(3.0, 10.0)	Fronteras de grupos en anos para <code>r_mode='grupos'</code> . Default: corto $\leq 3Y$, medio $\leq 10Y$, largo $> 10Y$.
plot	True	Si True, genera todos los graficos de diagnostico.

10.3. Estructura del retorno

Claves del dict que retorna `fit_dns_bayes()`

```

1 res = fit_dns_bayes(...)
2
3 # Muestras MCMC
4 res['chains']           # lista de n_chains dicts
5 res['priors']           # dict de priors usados
6 res['tau_mode']         # modo de tau utilizado
7
8 # Acceder muestras de una cadena:
9 ch = res['chains'][0]
10 ch['tau1']             # (n_store,) muestras de tau1 (estatico/ventana)
11 ch['tau1_seq']          # (n_store, T) trayectorias tau1_t (solo ar1)
12 ch['phi_x1']           # (n_store,) muestras phi_x (solo ar1)
13 ch['sigma_x1']         # (n_store,) muestras sigma_x (solo ar1)
14 ch['f_vec']            # (n_store, k) muestras diag(F)
15 ch['betas']            # (n_store, T, k) trayectorias de factores
16
17 # Diagnosticos
18 res['diagnostics']      # DataFrame con R-hat y ESS
19
20 # Ajuste in-sample (media posterior)
21 res['fit']['beta_mean']  # (T, k) factores  $E[\beta_t | Y]$ 
22 res['fit']['beta_q025']  # (T, k) percentil 2.5%
23 res['fit']['beta_q975']  # (T, k) percentil 97.5%
24 res['fit']['tau1_pm']    # float media posterior de tau1
25 res['fit']['tau1_t']     # (T,) secuencia tau1_t (ventana/ar1)

```

```

26 res['fit']['tau2_t']           # (T,)      secuencia tau2_t (DNSS)
27 res['fit']['tau_mode']        # str      modo de tau
28 res['fit']['fitted']          # (T, N)   rendimientos ajustados
29 res['fit']['residuals']       # (T, N)   residuos de medicion
30 res['fit']['rmse_total']      # float    RMSE total
31 res['fit']['rmse_mat']        # (N,)     RMSE por plazo

```

11. Ejemplos de uso

11.1. Ejemplo 1 — DNS basico (tau estatico)

Uso minimo --- DNS con priors por defecto

```

1 res = fit_dns_bayes(
2     dates      = df.index,
3     yields     = df[yield_cols].values,
4     maturities = np.array([1, 2, 3, 5, 10, 20], dtype=float),
5     model      = 'DNS',
6     n_iter     = 3000,
7     n_burnin   = 1000,
8     n_chains    = 4,
9     tau_mode   = 'estatico',    # comportamiento v1
10 )
11 print(f"RMSE = {res['fit']['rmse_total']*100:.3f} pb")
12
13 # Extraer factores
14 beta_t      = res['fit']['beta_mean']    # (T, 3)
15 nivel       = beta_t[:, 0]
16 pendiente   = beta_t[:, 1]
17 curvatura   = beta_t[:, 2]
18
19 # Distribucion posterior de tau1
20 tau1_samp = np.concatenate([ch['tau1'] for ch in res['chains']])
21 print(f"tau1: {tau1_samp.mean():.3f} "
22       f"IC95%=[{np.percentile(tau1_samp,2.5):.3f}, "
23       f"{np.percentile(tau1_samp,97.5):.3f}]")

```

11.2. Ejemplo 2 — DNSS con tau AR(1) latente

tau dinamico AR(1) --- DNSS

```

1 res = fit_dns_bayes(
2     dates      = df.index,
3     yields     = df[yield_cols].values,
4     maturities = mats,

```

```

5     model      = 'DNSS',
6     n_iter     = 5000,
7     n_burnin   = 2000,
8     n_chains   = 4,
9     tau_mode    = 'ar1',
10    tau_ar1_prop_std = 0.04,
11    prior_tau_type = 'normal',
12    tau1_mu=1.78, tau1_sd=0.40,
13    tau2_mu=0.60, tau2_sd=0.08,
14 )
15
16 # Secuencia temporal de tau1_t
17 tau1_t      = res['fit']['tau1_t']          # (T,) media posterior
18 tau1_seqs   = np.concatenate(
19     [ch['tau1_seq'] for ch in res['chains']], axis=0) # (S, T)
20 tau1_ic025  = np.percentile(tau1_seqs, 2.5, axis=0)
21 tau1_ic975  = np.percentile(tau1_seqs, 97.5, axis=0)
22
23 # Hiperparametros del proceso AR(1)
24 phi_samp    = np.concatenate([ch['phi_x1'] for ch in res['chains']])
25 sig_samp    = np.concatenate([ch['sigma_x1'] for ch in res['chains']])
26 print(f"phi_x:  {phi_samp.mean():.3f}  "
27       f"IC95%=[{np.percentile(phi_samp,2.5):.3f}, "
28       f" {np.percentile(phi_samp,97.5):.3f}]" )
29 print(f"sigma_x: {sig_samp.mean():.4f}  "
30       f"IC95%=[{np.percentile(sig_samp,2.5):.4f}, "
31       f" {np.percentile(sig_samp,97.5):.4f}]" )

```

11.3. Ejemplo 3 — DNS con tau por ventanas

tau semi-adaptativo por ventanas

```

1 res = fit_dns_bayes(
2     dates      = df.index,
3     yields     = df[yield_cols].values,
4     maturities = mats,
5     model      = 'DNS',
6     n_iter     = 2000,
7     n_burnin   = 600,
8     n_chains   = 4,
9     tau_mode    = 'ventana',
10    tau_window_size = 52,          # 52 periodos = ~1 ano (datos
        semanales)
11 )
12 tau1_t = res['fit']['tau1_t']      # (T,) tau1 por periodo
13 print(f"tau1 rango: [{tau1_t.min():.3f}, {tau1_t.max():.3f}]")
14 print(f"RMSE = {res['fit']['rmse_total']*100:.4f} pb")

```

11.4. Ejemplo 4 — Comparar los tres modos de tau

Comparativa automatica de los tres modos

```

1  # Una sola llamada corre y compara los 3 modos
2  resultados = comparar_modos_tau(
3      yields_data, maturities, dates_dt,
4      model      = 'DNS',
5      n_iter      = 2000,
6      n_burnin    = 600,
7      n_chains    = 2,
8      window_size = max(30, T // 5),
9      ar1_prop_std = 0.04,
10     verbose     = False,
11 )
12 # Imprime tabla comparativa RMSE / tau1 / R-hat
13 # Genera grafico plot_tau_temporal() con los 3 modos
14
15 # Acceso individual:
16 res_est      = resultados['Estatico']
17 res_ven      = resultados['Ventana']
18 res_ar1      = resultados['AR(1)']
19
20 # Grafico personalizado de evolucion de tau
21 plot_tau_temporal(resultados, dates_dt, maturities, titulo='DNS')

```

11.5. Ejemplo 5 — DNSS con Opciones A y C + tau ventana

Combinacion de mejoras para plazos largos y tau adaptativo

```

1  mats      = np.array([1, 2, 3, 5, 10, 20], dtype=float)
2  pesos     = np.where(mats >= 10, 2.0, 1.0)  # peso doble a 10Y y 20Y
3
4  res = fit_dns_bayes(
5      dates          = df.index,
6      yields         = df[yield_cols].values,
7      maturities     = mats,
8      model          = 'DNSS',
9      n_iter         = 4000,
10     n_burnin        = 1500,
11     n_chains        = 4,
12     tau_mode        = 'ventana',
13     tau_window_size = 60,
14     obs_weights     = pesos,          # Opcion C
15     r_mode          = 'grupos',      # Opcion A
16     group_thresholds = (3.0, 10.0),
17     prior_tau_type   = 'normal',
18     tau1_mu=1.78, tau1_sd=0.40,
19     tau2_mu=0.60, tau2_sd=0.08,

```

```

20 )
21 print(f"RMSE global: {res['fit']['rmse_total']*100:.4f} pb")
22 for i, m in enumerate(mats):
23     print(f"    {m:.0f}Y: {res['fit']['rmse_mat'][i]*100:.3f} pb")

```

11.6. Ejemplo 6 — Comparativa bayesiano vs KF

Comparar incertidumbre de factores: KF vs Bayes

```

1  # Suponer que res_kf es el resultado de fit_dns_kalman() (KF notebook)
2  # y res_bayes es el resultado de fit_dns_bayes()
3
4  # Comparar RMSE por plazo
5  for i, m in enumerate(maturities):
6      r_kf      = res_kf['rmse_mat'][i]          * 100
7      r_bayes   = res_bayes['fit']['rmse_mat'][i] * 100
8      print(f"    {m:.0f}Y  KF={r_kf:.3f}  Bayes={r_bayes:.3f}  "
9            f"Delta={r_bayes-r_kf:+.3f}")
10
11 # Ancho del IC de beta_0: KF (basado en P_smooth) vs Bayes (percentiles
    MCMC)
12 beta_mean, q025, q975, _ = extract_posterior_betas(res_bayes['chains'])
13 ancho_bayes = (q975[:,0] - q025[:,0]).mean()
14 ancho_kf     = (2*1.96*np.sqrt(res_kf['P_smoothed'][:,0,0])).mean()
15 print(f"\nIC beta0 --- KF: {ancho_kf:.4f}  Bayes: {ancho_bayes:.4f}")
16 print("(Bayes es mas amplio: integra incertidumbre de theta y tau)")

```

12. Referencia completa de funciones

Table 6: Todas las funciones del notebook v2 en orden de dependencia

Funcion	Retorno	Descripcion
ns_loadings(m, tau1)	$(N, 3)$	Cargas Nelson-Siegel
nss_loadings(m, tau1, tau2)	$(N, 4)$	Cargas Svensson
get_H(m, tau1, tau2, model)	(N, k)	Wrapper unificado
ols_init_tau(yields, m, model)	float, float	Inicio OLS two-step
_ffbs_diag(...)	(T, k)	FFBS Numba JIT
ffbs(...)	(T, k)	Wrapper publico FFBS
_kf_loglik_diag(...)	float	Log-lik marginal KF (JIT)
kf_loglik(...)	float	Wrapper log-lik
sample_F_mu_diagonal(...)	$(k,), (k,)$	Posterior F, μ
sample_Q(...)	$(k,)$	Posterior IG para Q
sample_R(...)	$(N,)$	Posterior IG para R libre
sample_R_groups(...)	$(N,)$	Posterior IG para R grupos
_log_prior_tau(...)	float	Log-prior de τ
sample_tau_MH_dns(...)	float, float, bool	MH 1D para τ_1
sample_tau_MH_dnss(...)	4 valores	MH 2D para (τ_1, τ_2)
make_priors(...)	dict	Priors (estatico/ventana)
make_priors_ar1(...)	dict	Priors + hiperpar. AR(1)
_ar1_loglik_tau_t(...)	float	Log-lik local para MH AR(1)
sample_tau_ar1_MH(...)	$(T, n_\tau), \text{float}$	MH por periodo para $x_{1:T}$
sample_phi_x(...)	$(n_\tau,)$	Posterior conj. ϕ_x
sample_sigma_x(...)	$(n_\tau,)$	Posterior conj. σ_x
initialize_chain(...)	dict	Inicializacion OLS
run_chain(...)	dict	Una cadena Gibbs (v2)
run_rolling_bayes(...)	list[dict]	Gibbs en ventanas rodantes
assemble_rolling_tau(...)	arrays	Ensambla τ_t de ventanas
run_chains_parallel(...)	list[dict]	Multi-cadena paralela
gelman_rubin(...)	float	Estadistico $\hat{R}$
effective_sample_size(...)	float	ESS
compute_diagnostics(...)	DataFrame	Tabla $\hat{R}$ y ESS
print_diagnostics(...)	—	Imprime diagnostics
plot_traceplots(...)	—	Traceplots por parametro
plot_acf_mcmc(...)	—	ACF con ESS
plot_posteriors(...)	—	Histogramas posteriores
plot_posterior_tau(...)	—	Posterior de τ y hump
extract_posterior_betas(...)	$(T, k) \times 4$	Media, Q2.5%, Q97.5%, std
plot_factores_bayes(...)	—	Factores con IC bayesiano
plot_tau_temporal(...)	—	Evolucion τ_t los 3 modos
compute_posterior_fit(...)	dict	Ajuste con media posterior
analisis_residuos_b(...)	—	Residuos con tests
plot_heatmap_residuos_b(...)	—	Heatmap residuos
comparar_modos_tau(...)	dict	Corre y compara 3 modos
fit_dns_bayes(...)	dict	Funcion principal v2

13. Guia de decisiones

Table 7: Que modo y configuracion usar segun el caso

Situacion	Configuracion recomendada	Justificacion
Exploracion rapida	<code>n_iter=1000, n_chains=2</code>	Verificar convergencia basica
Produccion / publicacion	<code>n_iter=5000+, n_chains=4</code>	$\hat{R} < 1.05$, ESS > 400
Curva estacionaria	<code>tau_mode='estatico'</code>	Mas rapido, IC de τ bien definido
Curva con cambios de regimen	<code>tau_mode='ventana'</code>	Equilibrio adaptabilidad / velocidad
Cambios graduales en τ_t	<code>tau_mode='ar1'</code>	Captura dinamica continua de la forma
Comparar los tres modos	<code>comparar_modos_tau(...)</code>	Tabla + grafico automatico
τ conocido a priori	<code>prior_tau_type='normal'</code>	Reduce exploracion innecesaria
Residuos grandes en largo	<code>obs_weights + r_mode='grupos'</code>	Opciones A y C combinadas
TES Colombia (2019–2026)	<code>tau_mode='ventana' o 'ar1'</code>	Multiples regimenes BanRep
Comparar con KF notebook	<code>plot_comparativa_bayes_kf()</code>	Visualiza ganancia en incertidumbre

Referencias

- Nelson, C.R. & Siegel, A.F. (1987). Parsimonious Modeling of Yield Curves. *Journal of Business*, 60(4), 473–489.
- Svensson, L.E.O. (1994). Estimating and Interpreting Forward Interest Rates: Sweden 1992–1994. *NBER Working Paper* No. 4871.
- Diebold, F.X. & Li, C. (2006). Forecasting the Term Structure of Government Bond Yields. *Journal of Econometrics*, 130(2), 337–364.
- Carter, C.K. & Kohn, R. (1994). On Gibbs Sampling for State Space Models. *Biometrika*, 81(3), 541–553.

-
- **Caldeira, J.F., Laurini, M.P. & Portugal, M.S.** (2010). Bayesian Inference Applied to Dynamic Nelson-Siegel Model. *Brazilian Review of Econometrics*, 30(1).
 - **Cakmaklı, C.** (2013). Modelling the Term Structure with a Bayesian Dynamic Nelson-Siegel Model. Erasmus University Rotterdam Working Paper.
 - **Kim, C.J. & Nelson, C.R.** (1999). *State-Space Models with Regime Switching*. MIT Press.
 - **Nakajima, J.** (2011). Time-Varying Parameter VAR Model with Stochastic Volatility. *IMES Discussion Paper Series* 2011-E-9.
 - **Gelman, A. & Rubin, D.B.** (1992). Inference from Iterative Simulation Using Multiple Sequences. *Statistical Science*, 7(4), 457–511.
 - **Roberts, G.O. & Rosenthal, J.S.** (2001). Optimal Scaling for Various Metropolis-Hastings Algorithms. *Statistical Science*, 16(4), 351–367.
 - **Hamilton, J.D.** (1994). *Time Series Analysis*. Princeton University Press. Capítulo 13: State-Space Models and the Kalman Filter.
 - **BIS** (2005). Zero-Coupon Yield Curves: Technical Documentation. *BIS Papers* No. 25.